

JOBSHEET 03

Encapsulation in Object-Oriented Programming

Ringga Budi Utama / 254107020251

1. Competence

After conducting experiments on this module, students understand the concept of:

1. Constructor
2. Access Modifier
3. Attributes/methods in a class
4. Attribute/method instantiation
5. Setter and getter
6. Understanding notation in a UML Class Diagram

2. Introduction

In the first and second meetings, the basic concepts of object-based programming (PBO), the difference between object-based programming and structural programming, were discussed, along with the concepts of classes and objects in PBO. In this module, the concept of encapsulation in PBO and the notation used in UML Class diagrams will be discussed.

2.1 Encapsulation

Encapsulation is also known as information-hiding. When interacting with objects, we often do not need to know the complexity that exists within them. This is easier to understand through everyday examples:

- Bicycle: when we change gears, we simply press the gear lever on the handlebar grip — we don't need to know how the gears move internally.
- Vacuum cleaner: when we plug in the cable and turn on the switch, the machine is ready to use — we don't need to know how electricity is converted into suction power.

In both examples, the object has implemented encapsulation (also known as information-hiding or data-hiding), because the internal process details are hidden from the user.

2.2 Constructors

A constructor is similar to a method declaration but has no return type. It is executed automatically whenever an instance of the object is created — every time an object is created using the `new()` keyword, the constructor runs. Rules for creating a constructor:

1. The constructor name must be the same as the class name.
2. The constructor does not have a return data type.
3. Constructors must not use the abstract, static, final, or synchronized modifiers.

Note: in Java, constructors may be declared private, protected, public, or with default (package-private) access. In general, constructors are declared public so the class can be freely instantiated by driver classes or other

external components. In more advanced OOP architecture, other modifiers are used to fulfil specific design needs:

Private constructor — restricts instantiation completely from outside the class. Commonly used in the Singleton Pattern (ensuring only one instance exists) or in Utility Classes containing only static methods (e.g. Java's built-in Math class), where object creation is irrelevant.

Default / protected constructor — restricts instantiation to classes within the same package, or allows initialization only by subclasses within an inheritance hierarchy.

2.3 Access Modifier

Java has two categories of modifiers: access modifiers and non-access modifiers. This section focuses on access modifiers, which control the visibility of methods, classes, and constructors. There are four access modifiers:

- **private** – accessible only within the same class
- **default** – accessible only within the same package
- **protected** – accessible outside the package, but only by subclasses (inheritance)
- **public** – accessible from anywhere

Table 1.1 Access Modifier Detail

Access Modifier	Notation	Within Class	Within Package	Outside Package (by Subclass only)	Outside Package
Private	– (minus)	Y	N	N	N
Default	~ (tilde)	Y	Y	N	N
Protected	# (hash)	Y	Y	Y	N
Public	+ (plus)	Y	Y	Y	Y

2.4 Getter and Setter

A getter is a public method with a return data type, used to retrieve the value of a private attribute. A setter is a public method with no return data type, used to manipulate (set) the value of a private attribute.

2.5 UML Class Diagram Notation

The general form of a UML class diagram is shown below.

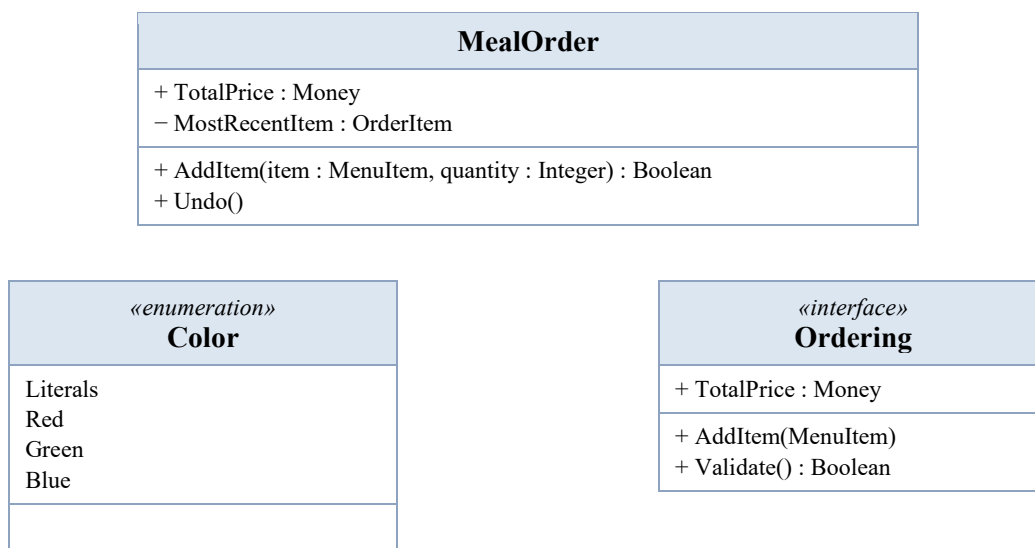


Figure 1.1 UML Class Diagram — general notation

Legend:

1. Class
2. Interface
3. Enumeration – a data type with a limited set of values (literals)
4. Attributes
5. Method
6. Literals

Access modifier notation used inside a UML class diagram:

1. Plus sign (+) for public
2. Hashtag (#) for protected
3. Minus sign (−) for private
4. Tilde sign (~), or no sign, for default

3. Experiments

3.1 Experiment 1 — Encapsulation

In this experiment, we create a Motor class that has speed and contactOn attributes, plus a printStatus() method to display the motor's state.

1. Open NetBeans and create a MotorEncapsulation project.
2. Create a Motorcycle class: right-click the motorencapsulation package → New → Java Class.
3. Type the Motor class code below.

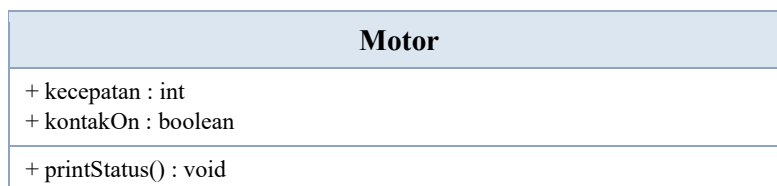
```
package motorencapsulation;

public class Motor {
    public int kecepatan = 0;
    public boolean kontakOn = false;

    public void printStatus(){
        if(kontakOn == true){
            System.out.println("Kontak On");
        }
        else{
            System.out.println("Kontak Off");
        }
        System.out.println("Kecepatan " + kecepatan+"\n");
    }
}
```

Motor.java — Experiment 1

The corresponding UML class diagram for the Motor class is as follows:



Next, create a MotorDemo class with the following code:

```
package motorencapsulation;
```

```

public class MotorDemo {
    public static void main(String[] args) {
        Motor motor = new Motor();
        motor.printStatus();
        motor.kecepatan = 50;
        motor.printStatus();
    }
}

```

MotorDemo.java — Experiment 1

The output is as follows:

```

run:
Kontak Off
Kecepatan 0

Kontak Off
Kecepatan 50

BUILD SUCCESSFUL (total time: 0 seconds)

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'motorencapsulation.MotorDemo'
Kontak Off
Kecepatan 0

Kontak Off
Kecepatan 50

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

From Experiment 1, is anything wrong here? The motor's speed suddenly jumps from 0 to 50 — and even stranger, the ignition (contact) is still OFF. How can a motorcycle accelerate from zero to 50 instantly while the ignition is off?

Access to the motorcycle's attributes is uncontrolled. In the real world, objects always have limitations and mechanisms governing how they can be used. To improve the Motorcycle class so it behaves properly, we should:

1. Hide internal attributes (speed, contactOn) from other classes.
2. Provide dedicated methods for accessing those attributes.

For that, let's continue with the next experiment on Access Modifiers.

3.2 Experiment 2 — Access Modifier

In this experiment, access modifiers are used to improve how the Motor class works, based on the following UML class diagram.

Motor
- kecepatan : int - kontakOn : boolean
+ nyalakanMesin() : void + matikanMesin() : void + tambahKecepatan() : void + kurangiKecepatan() : void + printStatus() : void

Changes made to the Motor class:

- Change the access modifier of kecepatan and kontakOn to private.
- Add the methods nyalakanMesin, matikanMesin, tambahKecepatan, and kurangiKecepatan.

Implementation of the Motor class:

```
package motorencapsulation;

public class Motor {
    private int kecepatan = 0;
    private boolean kontakOn = false;    // access modifier changed to private

    public void nyalakanMesin(){
        kontakOn = true;
    }
    public void matikanMesin(){
        kontakOn = false;
        kecepatan = 0;
    }
    public void tambahKecepatan() {        // method added
        if (kontakOn == true){
            kecepatan += 5;
        }
        else{
            System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off! \n");
        }
    }
    public void kurangiKecepatan() {        // method added
        if (kontakOn == true){
            kecepatan -= 5;
        }
        else{
            System.out.println("Kecepatan tidak bisa berkurang karena Mesin Off! \n");
        }
    }
    public void printStatus(){
        if (kontakOn == true){
            System.out.println("Kontak On");
        }
        else{
            System.out.println("Kontak Off");
        }
        System.out.println("Kecepatan " + kecepatan+"\n");
    }
}
```

Motor.java — Experiment 2

The MotorDemo class is updated as follows:

```
package motorencapsulation;

public class MotorDemo {
    public static void main(String[] args) {
        Motor motor = new Motor();
        motor.printStatus();
        motor.tambahKecepatan();

        motor.nyalakanMesin();
        motor.printStatus();

        motor.tambahKecepatan();
        motor.printStatus();

        motor.tambahKecepatan();
        motor.printStatus();
    }
}
```

```

        motor.tambahKecepatan();
        motor.printStatus();

        motor.matikanMesin();
        motor.printStatus();
    }
}

```

MotorDemo.java — Experiment 2

The output is as follows:

```

run:
Kontak Off
Kecepatan 0

Kecepatan tidak bisa bertambah karena Mesin Off!

Kontak On
Kecepatan 0

Kontak On
Kecepatan 5

Kontak On
Kecepatan 10

Kontak On
Kecepatan 15

Kontak Off
Kecepatan 0

BUILD SUCCESSFUL (total time: 0 seconds)

```

```

_ws\Jobsheet 3 Pbo_8696bda1\bin' 'motorencapsulation.MotorDemo'
Kontak Off
Kecepatan 0

Kecepatan tidak bisa bertambah karena Mesin Off!

Kontak On
Kecepatan 0

Kontak On
Kecepatan 5

Kontak On
Kecepatan 10

Kontak On
Kecepatan 15

Kontak Off
Kecepatan 0

```

From this experiment, the speed attribute can no longer be changed carelessly by the user. Even trying to increase the speed while the ignition is OFF produces a notification that the engine is off. To reach the desired speed, `tambahKecepatan()` must be called repeatedly — much like riding a real motorcycle.

3.3 Questions

1. In the `MotorDemo` class, when we increase the speed for the first time, why does the warning "Kecepatan tidak bisa bertambah karena Mesin Off!" appear?

Answer:

Karena nilai awal variabel `kontakOn` diset false (OFF). Method `tambahKecepatan()` memiliki validasi kondisi `if (kontakOn == true)`. Jika mesin belum dinyalakan, nilai `kontakOn` masih false sehingga sistem memblokir penambahan kecepatan.

2. Why are the `kecepatan` and `kontakOn` attributes set to private?

Answer:

Untuk menerapkan konsep **encapsulation** (informasi/data hiding). Hal ini mencegah kelas luar mengubah atribut secara langsung secara sembarangan tanpa melalui method kontrol yang valid.

3. Modify the `Motor` class so that the maximum speed is 100.

Answer:

```
public void tambahKecepatan() {
    if (kontakOn == true) {
        if (kecepatan + 5 <= 100) {
            kecepatan += 5;
        } else {
            kecepatan = 100;
            System.out.println("Kecepatan maksimal telah tercapai (100)!\n");
        }
    } else {
        System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off! \n");
    }
}
```

3.4 Experiment 3 — Getter and Setter

Consider a cooperative (koperasi) information system with a Member (Anggota) class. A member has the attributes nama, alamat, and simpanan, and methods for setter, getter, deposit (setor), and borrow (pinjam). No attribute should be changed indiscriminately — only through the setter, getter, deposit, and draw methods. The simpanan attribute has no setter, since it only changes through deposit() and borrow/withdraw() transactions.

UML class diagram for the Anggota class:

Anggota
– nama : String – alamat : String – simpanan : float
+ setName(String) : void + setAlamat(String) : void + getName() : String + getAlamat() : String + getSimpanan() : float + setor(float) : void + pinjam(float) : void

Create a new project, following the same process as Experiment 1:

- Open NetBeans, create a GetterSetterCooperative project.
- Create an Anggota class: right-click the koperasigettersetter package → New → Java Class.
- Type in the Anggota class code below.

```
package koperasigettersetter;

public class Anggota {
    private String nama;
    private String alamat;
    private float simpanan;

    public void setName(String nama){        // setter, getter
        this.nama = nama;                    // nama dan alamat
    }
    public void setAlamat(String alamat){
        this.alamat = alamat;
    }
    public String getName(){
        return nama;
    }
    public String getAlamat(){
        return alamat;
    }
    public float getSimpanan(){              // getter simpanan
        return simpanan;
    }
    public void setor(float uang){
        simpanan += uang;
    }
    public void pinjam(float uang){
        simpanan -= uang;
    }
}
```

Anggota.java — Experiment 3

Notice that the nama and alamat attributes each have one getter and one setter, while simpanan only has getSimpanan() — since, per the original design, its value only changes through setor() and pinjam() transactions.

Next, create a KoperasiDemo class to try out the Anggota class:

```

package koperasigettersetter;

public class KoperasiDemo {
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota();
        anggota1.setNama("Iwan Setiawan");
        anggota1.setAlamat("Jalan Sukarno Hatta no 10");
        anggota1.setor(100000);
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

        anggota1.pinjam(5000);
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
    }
}

```

KoperasiDemo.java — Experiment 3

The output of the main method is as follows:

```

run:
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hot
spot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\U
ser\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'koperasigetters
etter.KoperasiDemo'
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

As shown, the deposit (simpanan) is never changed directly — it is always changed through the setor() and pinjam() methods. Likewise, the name is only displayed via getNama(), and the balance via getSimpanan().

3.5 Experiment 4 — Constructors

Step 1: change the KoperasiDemo class as follows:

```

package koperasigettersetter;

public class KoperasiDemo {
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota();
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

        anggota1.setNama("Iwan Setiawan");
        anggota1.setAlamat("Jalan Sukarno Hatta no 10");
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

        anggota1.pinjam(5000);
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
    }
}

```

Result of the program:

```

run:
Simpanan null : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

```

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> c:: cd 'c:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'koperasigettersetter.KoperasiDemo'
Simpanan Iwan : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

Notice that calling `getNama()` before `setNama()` returns null, because the name attribute has not yet been set to a default value. This is resolved by adding a constructor.

Update the `Anggota` class as follows:

```

package koperasigettersetter;

public class Anggota {
    private String nama;
    private String alamat;
    private float simpanan;

    Anggota(String nama, String alamat){
        this.nama = nama;
        this.alamat = alamat;
        this.simpanan = 0;
    }

    public void setNama(String nama){
        this.nama = nama;
    }
    public void setAlamat(String alamat){
        this.alamat = alamat;
    }
    public String getNama(){
        return nama;
    }
    public String getAlamat(){
        return alamat;
    }
    public float getSimpanan(){
        return simpanan;
    }
    public void setor(float uang){
        simpanan += uang;
    }
    public void pinjam(float uang){
        simpanan -= uang;
    }
}

```

Anggota.java — Experiment 4 (with constructor)

A constructor with default access is added, taking `nama` and `alamat` as parameters, and initializing `simpanan` to 0.

Next, update the `KoperasiDemo` class as follows (passing parameters):

```

package koperasigettersetter;

public class KoperasiDemo {
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota("Iwan", "Jalan Mawar"); // passing parameters
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

        anggota1.setNama("Iwan Setiawan");
        anggota1.setAlamat("Jalan Sukarno Hatta no 10");
    }
}

```

```

    anggota1.setor(100000);
    System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

    anggota1.pinjam(5000);
    System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
}
}

```

KoperasiDemo.java — Experiment 4

Result of the program:

```

run:
Simpanan Iwan : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
● PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> c:; cd 'c:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'koperasigettersetter.KoperasiDemo'
Simpanan Iwan : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
○ PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

After adding a constructor to the Anggota class, the nama and alamat attributes must automatically be set first, by passing parameters, when instantiating the Anggota class. This is typically done for attributes that require specific starting values. If a specific value isn't required, it doesn't need to be a constructor parameter — for example, a new member's simpanan is always set to 0, so it is not included as a constructor parameter.

3.6 Questions — Experiments 3 and 4

1. What do getters and setters mean?

Answer:

1. **Getter:** Method public ber-return type yang berfungsi untuk membaca/mengambil nilai atribut private.
2. **Setter:** Method public ber-return void yang berfungsi untuk mengisi/mengubah nilai atribut private.

2. What is the use of the getSimpanan() method?

Answer:

Untuk mengambil dan mengembalikan nilai dari atribut simpanan yang bersifat private

3. What method is used to top up (deposit into) the balance?

Answer:

Method setor(float uang).

4. What does a constructor mean?

Answer:

Method khusus dalam kelas yang dipanggil secara otomatis saat objek diinstansiasi menggunakan kata kunci new untuk menginisialisasi atribut/state awal objek

5. What are the rules for creating constructors?

Answer:

1. Nama constructor harus sama persis dengan nama kelas.
2. Tidak memiliki tipe data kembalian (return type), bahkan void.
3. Tidak boleh menggunakan modifier abstract, static, final, atau synchronized.

6. Is it acceptable for a constructor to have private access?

Answer:

Boleh. Biasanya digunakan pada pola desain *Singleton Pattern* atau *Utility Classes* untuk mencegah instansiasi objek dari luar kelas.

7. When should parameters be passed via a constructor?

Answer:

Saat suatu objek membutuhkan nilai awal tertentu yang wajib diisi ketika objek pertama kali dibuat

8. What is the difference between a class attribute and an instance attribute?

Answer:

1. **Atribut Instance:** Milik dari objek/instance individual (tiap objek punya salinan nilai sendiri).
2. **Atribut Kelas (static):** Milik bersama seluruh kelas dan nilainya terbagi untuk semua instance.

9. What is the difference between a class method and an instance method?

Answer:

1. **Method Instance:** Bekerja pada data instance dan dipanggil melalui objek instance.
2. **Method Kelas (static):** Bekerja secara independen dari instance dan dipanggil langsung melalui nama kelasnya.

4. Conclusion

From the experiments above, the concepts of encapsulation, constructors, and access modifiers have been studied. There are four access modifier types: public, protected, default, and private. We also studied the concept of attributes and methods within a class block, the concept of instantiating attributes/methods, how to use getters and setters, and the meaning of UML notation.

5. Tasks

1. Try the program below and write down the output.

```
public class EncapDemo
{
    private String name;
    private int age;

    public String getName()
    {
        return name;
    }

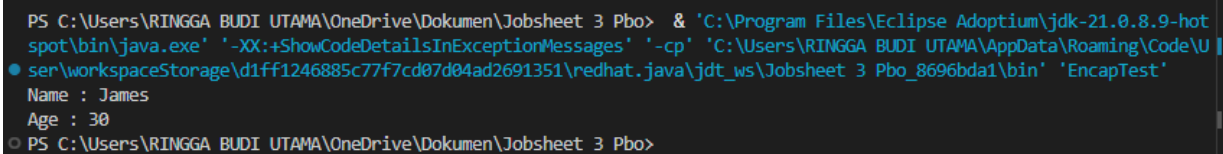
    public void setName(String newName)
    {
        name = newName;
    }

    public int getAge()
    {
        return age;
    }

    public void setAge(int newAge)
    {
        if(newAge > 30)
        {
            age = 30;
        }
        else
        {
            age = newAge;
        }
    }
}
```

```
public class EncapTest
{
    public static void main(String args[])
    {
        EncapDemo encap = new EncapDemo();
        encap.setName("James");
        encap.setAge(35);

        System.out.println("Name : " + encap.getName());
        System.out.println("Age : " + encap.getAge());
    }
}
```

Output:


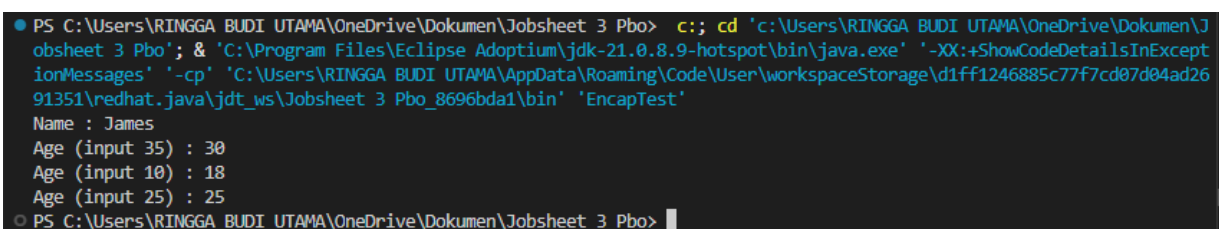
```
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'EncapTest'
Name : James
Age : 30
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>
```

2. In the program above, in the EncapTest class we set the age to 35, but when it is displayed on screen the value is 30. Explain why.

Answer:

Meskipun pada EncapTest diisi `encap.setAge(35)`, method `setAge` pada EncapDemo memiliki logika pemeriksaan `if (newAge > 30)`. Karena nilai 35 lebih besar dari 30, nilai variabel age secara otomatis dipatok menjadi 30.

3. Change the program above so that the age attribute can be given a maximum value of 30 and a minimum of 18.

Answer:


```
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> c:; cd 'c:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'EncapTest'
Name : James
Age (input 35) : 30
Age (input 10) : 18
Age (input 25) : 25
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>
```

4. Container (Kontainer) logistics problem

In an expedition cargo warehouse management system, there is a *Kontainer* class with attributes including *nomorResi*, *namaPemilik*, *kapasitasMaksimal* (in kg), and *beratMuatanSaatIni*. A container can accept additional cargo weight up to its specified maximum capacity limit. Cargo can also be unloaded from the container; when this happens, the current load weight decreases according to the weight removed.

Create the *Kontainer* class with private attributes, getter methods, and constructors according to encapsulation architecture requirements. Test it with the following *TestLogistik* driver class:

```
public class TestLogistik {
    public static void main(String[] args) {
        Kontainer kontainerAlfa = new Kontainer("REQ-9988", "PT. Maju Bersama", 5000);

        System.out.println("Nama Pemilik Kontainer: " + kontainerAlfa.getNamaPemilik());
        System.out.println("Kapasitas Maksimal: " + kontainerAlfa.getKapasitasMaksimal() + " kg");

        System.out.println("\nMemasukkan muatan baru seberat 6.000 kg...");
        kontainerAlfa.tambahMuatan(6000);
        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");

        System.out.println("\nMemasukkan muatan baru seberat 4.000 kg...");
        kontainerAlfa.tambahMuatan(4000);
        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");

        System.out.println("\nMembongkar muat/menurunkan barang seberat 500 kg...");
        kontainerAlfa.turunkanMuatan(500);
        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");

        System.out.println("\nMembongkar muat/menurunkan barang seberat 1.500 kg...");
        kontainerAlfa.turunkanMuatan(1500);
        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
    }
}
```

TestLogistik.java

Expected output:

```
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg
```

```

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> & 'C:\Program Files\Eclipse
spot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI
ser\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_

Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

Answer:

```
import java.util.Scanner;
```

```

public class TestLogistik {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        Kontainer kontainerAlfa = new Kontainer("KTR-9988", "PT Maju Bersama", 3000);

        System.out.println("Nama Pemilik Kontainer : " + kontainerAlfa.getNamaPemilik());

        System.out.println("Kapasitas Maksimal    : " + kontainerAlfa.getKapasitasMaksimal() + " kg\n");

        System.out.print("Masukkan berat muatan yang akan ditambah (kg): ");

        double tambah = input.nextDouble();

        kontainerAlfa.tambahMuatan(tambah);

        System.out.println("Berat muatan saat ini : " + kontainerAlfa.getBeratMuatanSaatIni() + " kg\n");

        System.out.print("Masukkan berat muatan yang akan diturunkan (kg): ");
    }
}

```

```

double turun = input.nextDouble();

kontainerAlfa.turunkanMuatan(turun);

System.out.println("Berat muatan saat ini : " + kontainerAlfa.getBeratMuatanSaatIni() + " kg\n");

input.close();
}
}

```

5. Enforce a 50% safety limit on unloading

Modify the cargo logistics problem above so that the weight unloaded in a single call to `turunkanMuatan()` is allowed to be a maximum of only 50% of the current total load weight. This is implemented for operational safety reasons regarding heavy equipment (cranes). If an operator attempts to unload cargo exceeding this 50% limit, the system must block the action and display the warning: "Sorry, for safety reasons, a single cargo unloading operation cannot exceed 50% of the current load!"

Answer:

```

PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> c:: cd 'c:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\AppData\Roaming\Code\User\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo_8696bda1\bin' 'TestLogistik'
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Mencoba menurunkan 2.500 kg (lebih dari 50% dari 4000 kg)...
Sorry, for safety reasons, a single cargo unloading operation cannot exceed 50% of the current load!
Berat muatan saat ini: 4000.0 kg

Menurunkan 1.000 kg (masih dalam batas 50%)...
Berat muatan saat ini: 3000.0 kg
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

6. Add interactive input with Scanner

Modify the driver class `TestLogistik` so that the weight parameter values entered for adding cargo (`tambahMuatan`) and unloading cargo (`turunkanMuatan`) can accept dynamic user input interactively through the terminal, using the `java.util.Scanner` utility.

Answer:

```

91551\Feunat.java\Jut_ws\Jobsheet 3 PBO\8090001\bin TestLogistik
Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Masukkan berat muatan yang ingin ditambahkan (kg): 90
Berat muatan saat ini: 90.0 kg

Masukkan berat muatan yang ingin diturunkan (kg): 90
Berat muatan saat ini: 0.0 kg
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo>

```

7. Movie ticket booking — Tiket class

A movie ticket booking application requires a Tiket class to manage booking data securely. This class must have the following private attributes: judulFilm (String), hargaDasar (double), and statusPembayaran (boolean).

Object value setup requirements:

- The constructor must accept parameters for judulFilm and hargaDasar. The initial value of statusPembayaran is always set to false (Unpaid).
- The hargaDasar attribute cannot be negative. If the provided input is less than 0, automatically set it to a default value of Rp 35,000.
- Provide a lakukanPembayaran() method to change statusPembayaran to true.
- The statusPembayaran value must be read-only via a getter, and must not have a direct setter from outside the class, for transaction security reasons.

Test your code using the following TestBioskop class:

```

public class TestBioskop {
    public static void main(String[] args) {
        Tiket tiket1 = new Tiket("Avengers: Endgame", -50000);
        System.out.println("Film: " + tiket1.getJudulFilm());
        System.out.println("Harga Tiket: " + tiket1.getHargaDasar());
        System.out.println("Status Lunas? " + tiket1.isStatusPembayaran());

        System.out.println("\nMemproses pembayaran...");
        tiket1.lakukanPembayaran();
        System.out.println("Status Lunas Terbaru? " + tiket1.isStatusPembayaran());
    }
}

```

TestBioskop.java

Answer:

```

public class Tiket {

    private String judulFilm;

```

```
private double hargaDasar;

private boolean statusPembayaran;

public Tiket(String judulFilm, double hargaDasar) {
    this.judulFilm = judulFilm;
    if (hargaDasar < 0) {
        this.hargaDasar = 35000;
    } else {
        this.hargaDasar = hargaDasar;
    }
    this.statusPembayaran = false;
}

public String getJudulFilm() {
    return judulFilm;
}

public double getHargaDasar() {
    return hargaDasar;
}

public boolean isStatusPembayaran() {
    return statusPembayaran;
}

public void lakukanPembayaran() {
    this.statusPembayaran = true;
}
}
```

```
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\RINGGA BUDI UTAMA\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo' 'C:\Users\RINGGA BUDI UTAMA\workspaceStorage\d1ff1246885c77f7cd07d04ad2691351\redhat.java\jdt_ws\Jobsheet 3 Pbo\Film: Avengers: Endgame  
Harga Tiket: 35000.0  
Status Lunas? false  
  
Memproses pembayaran...  
Status Lunas Terbaru? true  
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Jobsheet 3 Pbo> |
```